

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

电控组第三讲

培训时间：第 8 周 10.24 主讲人：_____



周次与日期	培训内容	本节重点	课后成果
军训培训 9月25日	C语言①：环境配置与基础语法	编译运行过程、变量、数据类型、常量、运算符、输入输出、类型转换	能独立创建工程并完成简单计算程序
第7周 10月17日	C语言②：程序流程与函数，数组、字符串、结构体与指针	if、switch、for、while、函数、参数、返回值、作用域、数组下标、字符串结尾、结构体组织数据、地址与指针、指针传参	掌握分支语句、结构体、指针等高阶语句，初步掌握C语言
第8周 10月24日	STM32①：GPIO、按键与外部中断	工程建立、GPIO输入输出、LED、上下拉、EXTI、按键消抖	完成按键控制LED，并能处理按键抖动
第9周 10月31日	STM32②：定时器、PWM	定时器周期计算、非阻塞闪灯、PWM频率与占空比、舵机脉宽控制	完成PWM控制LED呼吸灯与舵机角度
第10周 11月7日	STM32③：UART与I ² C OLED	串口收发、字符串转数字、命令解析；OLED初始化和数据显示	串口控制频率/占空比/舵机角度，OLED显示系统状态
第11周 11月14日	正式考核	独立完成、录制视频演示、代码内容讲解	在安排的规定考核时段内完成，周日前提交



目录

01

工程建立与 GPIO 输出

02

GPIO 输入与按键

03

按键消抖

04

外部中断 EETI

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

01 工程建立与 GPIO 输出



① 启动文件与链接脚本

C 程序从 main 开始，单片机却是先跑 startup 汇编：它设置栈指针、初始化时钟，再跳进 main。

② HAL 库

HAL_GPIO_TogglePin() 内部帮你算好了寄存器地址与位号，不用自己查手册写十六进制。

③ 下载器与 Flash

ST-Link 把编译好的 bin 烧进 Flash，掉电不丢；单步调试靠 SWD 两根线。

```
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init(); // CubeMX 生成
    while (1) {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
        HAL_Delay(500);
    }
}
```

C 负责逻辑，HAL 负责接口，工程负责底座——三者齐了，灯才会亮。



① 选芯片、定时钟

选 STM32F103C8T6，外部晶振 8MHz 倍频到 72MHz——时钟不对，串口和定时器全是错的。

② 配引脚、命名

PC13 点成 GPIO_Output，模式选推挽输出、无上下拉；引脚改名 LED 以后好找。

③ 生成、只改 USER CODE

Toolchain 选 MDK-ARM；自己写的代码放在 USER CODE BEGIN 与 END 之间，重新生成不会被冲掉。

```
void MX_GPIO_Init(void) {  
    GPIO_InitTypeDef g = {0};  
    g.Pin = GPIO_PIN_13;  
    g.Mode = GPIO_MODE_OUTPUT_PP;  
    g.Pull = GPIO_NOPULL;  
    g.Speed = GPIO_SPEED_FREQ_HIGH;  
    HAL_GPIO_Init(GPIOC, &g);  
}
```

CubeMX 把寄存器配置翻译成 init 代码，你的工作只剩 while(1) 里那几行逻辑。



① WritePin 定电平

三个参数：哪个口、哪根脚、给高还是给低。PC13 是低电平点亮，写 RESET 才亮。

② TogglePin 翻转

翻转不用记当前电平，配一次 Delay 就是闪烁；想快想慢，只改 Delay 的毫秒数。

③ Delay 的代价

HAL_Delay 是空转等待，等待期间什么都干不了——灯闪了，按键就顾不上，这是真实问题。

```
while (1) {  
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13,  
        GPIO_PIN_RESET);  
    HAL_Delay(1000);  
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13,  
        GPIO_PIN_SET);  
    HAL_Delay(1000);  
}
```

想一想：两个 HAL_Delay(1000) 都留着，灯是 1 秒闪一次；去掉第二个，灯会怎样？作业里有这道题。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

02 GPIO 输入与按键



① 上拉输入：平时 1，按下 0

引脚内部经电阻接 VCC，按键另一端接地。松手读到高电平，按下被拉低——最常用接法。

② 下拉输入：平时 0，按下 1

电阻接 GND，按下接 VCC。判断逻辑要跟着反过来写，接反了灯会一直亮或一直不亮。

③ 浮空输入：绝对不能用

没有电阻定电平，引脚像根天线，读到的值随手指和空气乱跳，程序无法判断。

```
// CubeMX: PB1 -> GPIO_Input
// Pull-up -> 平时读 1
if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_1)
    == GPIO_PIN_RESET) {
    HAL_GPIO_WritePin(GPIOC,
        GPIO_PIN_13, GPIO_PIN_RESET);
}
```

「按下读到 0 还是 1」由 CubeMX 里的上拉 / 下拉决定，代码只负责如实读出来。



模式	松手时电平	按下时电平	能不能用在按键上
上拉输入	1（高）	0（低）	推荐，作业里就用它
下拉输入	0（低）	1（高）	可用，判断条件要反过来写
浮空输入	不确定	不确定	不要用，读数会乱跳
模拟输入	连续电压	连续电压	给 ADC 采样用，不接按键

同一个按键配上拉和配下拉，代码里判断的电平正好相反——选错模式，现象就是灯不亮或长亮。



① 判断电平：现在按着吗

ReadPin 等于 RESET 就是按着。按住不放它会一直成立，灯会一直翻，不是「按一下翻一次」。

② 判断边沿：刚刚按了一下

存下上一次的电平，只有「上次 1、这次 0」才算一次按下——这才是按一下动一次。

③ 区分短按与长按

要计时就得有「现在几点」，HAL_GetTick 能凑合，第 9 周的定时器才是正规做法。

```
static uint8_t last = 1;
uint8_t now = HAL_GPIO_ReadPin(GPIOB,
GPIO_PIN_1);
// 下降沿：刚刚按下
if (last == 1 && now == 0) {
    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
}
last = now;
```

判断电平回答「现在按着吗」，判断边沿回答「刚刚按了一下」——两个需求不能混在同一行 if 里。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

03 按键消抖



① 抖动是机械问题

触点闭合的几毫秒内会反复通断，不是芯片的错；CPU 一毫秒能读几百次，全都读到了。

② 延时消抖：20ms 后再确认

读到按下后等 20 毫秒再读一次，两次都是按下才认账。简单有效，但 Delay 会挡住别的事。

③ 状态机消抖：不阻塞

用时间戳和状态变量判断，不等不卡；等第 9 周定时器到位，这才是工程里的标准写法。

```
uint8_t now = ReadKey();
if (now == 0 && last == 1) {
    HAL_Delay(20);
    if (ReadKey() == 0) { // 再确认一次
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
    }
}
last = now;
```

抖动的本质是「一次按下被读成很多次」；消抖就是让这很多次只算一次。



方案	怎么判断	代价	用在哪
延时消抖	等 20ms 再读一次确认	主循环停 20ms	作业、单个按键
计数消抖	连续 N 次读到相同才认	次数不好定	循环稳定的简单场合
状态机消抖	用时间戳判稳定时长	代码稍长，要有时间基准	工程里的标准做法
硬件消抖	RC 电路或专用芯片先滤	加器件、占板面	按键多、CPU 忙の場合

先问一句：主循环里除了按键还有别的事吗？只有按键，延时消抖就够；还有别的事，必须换成状态机或硬件。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

04 外部中断 EETI



① 定触发边沿

PB1 在 CubeMX 里选 GPIO_EXTI1，按键接地配下降沿：松手是 1，按下瞬间跳成 0 就触发。

② NVIC 里使能中断

光配引脚不勾 NVIC，中断永远不会进——报错没有，只是没反应，这个坑非常常见。

③ 逻辑写进回调

所有 EXTI 共用一个回调，先用 GPIO_Pin 判断是谁触发的，再决定做什么。

```
// USER CODE 区里自己写
void HAL_GPIO_EXTI_Callback(uint16_t
GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_1) {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
    }
}
```

配置在 CubeMX 里做完，处理逻辑写进回调——回调里只做最短的事，长活儿交回主循环。



① 能做的：几微秒的事

翻转一个引脚、把一个标志位置 1、存一个计数——做完立刻退出，这才是中断该干的事。

② 不能做的：等待与打印

HAL_Delay、长循环、串口打印都要占时间，会让别的中断排队，严重时程序像死机。

③ 共享变量的坑

中断和主循环都要读写的变量必须加 volatile，否则编译器可能认为它没变过，把它优化掉。

```
volatile uint8_t key_flag = 0;
void HAL_GPIO_EXTI_Callback(uint16_t pin)
{
    if (pin == GPIO_PIN_1) key_flag = 1; //
    只记一笔
}
// 主循环里慢慢处理
if (key_flag) { key_flag = 0; /* 干重活
*/ }
```

中断只负责「记一笔」，主循环负责「忙一阵」——分工清楚，程序才不会被一次按键卡住。



对比项	轮询（while 里读）	中断（EXTI 通知）	结论
CPU 占用	每一圈都问一遍	只有事件来了才跑	中断省时间
响应速度	看循环一圈多快	微秒级，立刻进回调	中断更及时
代码难度	直观，一眼看懂	要配 NVIC、注意共享变量	轮询好写
适合场景	按键少、主循环空闲	任务多、要省电	按需求选

两个问题定方案：CPU 还有别的事要做吗？这个事件有多急？——答不上来，说明需求还没想清。



01

解释现象

两个 HAL_Delay(1000) 都保留时，灯 1 秒闪一次；去掉第二个，灯会一直亮。先说清为什么，再动手改一遍验证。

02

CubeMX 配置按键控灯

用 CubeMX 配置一个按键，控制 PC13 上的灯：按下点亮，再按熄灭。

03

用 EXTI 实现

用 PB1 做外部中断，控制 PA1 上的 LED 亮灭，并在代码里用注释说明整个执行流程。

04

换一根引脚再来一遍

换成 PA11 做外部中断，同样控制 PA1 上的 LED——换引脚时要改哪些地方，自己总结。

四道题正好对应本节四块内容，做完再上传；代码注释必须统一 UTF-8，否则别人打开全是乱码。



01

代码注释必须统一 UTF-8 ；交上来打不开、或者注释是乱码的，按未提交处理。

02

演示视频、代码和其他材料放在同一个压缩包里，不要拆成几个文件分开发。

03

压缩包命名格式：班级 + 姓名 + 第几次任务，例如「电气 26-5 班 + 李华 + 第一次任务 .zip」。

04

发到指定邮箱，邮件主题与压缩包同名，压缩包作为附件上传，不要用网盘链接。

05

不强制每周内完成，但转入梯队考核上传结束时间前必须全部交齐（第 11 周）。

五条要求都是硬性的：代码写得对，但包名写错、注释乱码，一样按未提交处理。

『生无所息，斗无所止』



HUST Artificial Intelligence Robot laboratory

哈尔滨理工大学AIR人工智能机器人创新实验室

下课啦!